

Задача: «Симулятор автономных модулей на фрагментированной карте»

1. Контекст и постановка

Вы разрабатываете ядро пошаговой симуляции для системы, в которой автономные модули перемещаются по нерегулярной (фрагментированной) карте. Карта представлена **зубчатым массивом**, так как реальная область покрытия часто имеет рваные границы (например, сектора разной ширины). Каждый модуль обладает уникальной логикой поведения, расходом ресурсов и правилами перемещения. Ваша задача реализовать объектную модель, арифметику координат и движок обновления состояния, используя только изученные конструкции.

2. Требования к реализации

2.1. Структура координат `Coord`

- `struct` с полями `X`, `Y` (тип `int`).
- Перегрузка операторов: `+`, `-`, `*` (умножение на скаляр `int`), `==`, `!=`.
- **Обязательно:** при перегрузке `==` и `!=` необходимо переопределить `Equals(object)` и `GetHashCode()`. (Без этого сравнение ссылочного типа с переопределённым `==` приведёт к несоответствию поведения в хэш-коллекциях и нарушит контракт `System.Object`.)
- Рекомендуется объявить `readonly struct` для гарантии неизменяемости и безопасной передачи через `in`.

2.2. Базовый класс модуля `AbstractModule`

- `abstract class`, содержит:
 - `protected Coord Position`;
 - `protected int Fuel`;
 - `abstract void Act()`;
 - `virtual void ConsumeFuel(int amount)` (по умолчанию уменьшает `Fuel`, но не ниже 0).
- Конструктор должен инициализировать `Position` и `Fuel`.

2.3. Конкретные модули

- `sealed class ScoutModule : AbstractModule`
 - `Act()`: перемещает модуль на (1, 0) или (0, 1) (выбор на ваше усмотрение), тратит 1 единицу топлива.
- `sealed class CargoModule : AbstractModule`
 - `Act()`: перемещает модуль на (0, -1) или (-1, 0), тратит 2 единицы топлива.
- Оба класса должны быть `sealed`, чтобы запретить дальнейшее наследование (гарантия закрытости контракта и возможность JIT-оптимизации виртуальных вызовов).

2.4. Движок симуляции SimulationEngine

- Содержит методы, демонстрирующие работу с ref, in, out:

1. `public bool TryStep(AbstractModule[][] map, in int turnNumber, out int processedCount, ref int totalFuelConsumed)`

- Проходит по зубчатому массиву.
- Для каждого не-null модуля вызывает `Act()`.
- Обновляет `totalFuelConsumed` (через `ref`).
- Возвращает количество обработанных модулей через `out processedCount`.
- `in turnNumber` передаётся только для чтения (например, для логирования или детерминированного смещения).

2. `public static void ShiftMapCoords(AbstractModule[][] map, in Coord offset)`

- Применяет смещение координат ко всем модулям, используя перегруженный `+`.